Practical: 9

Date:08/10/2026

Hyperparameter Optimization and Model Selection

For digits dataset in sklearn. dataset, try the following classifiers and find out the one that gives best performance. Also find the optimal parameters for that classifier.

- **Svm**
- **Random Forest**
- **Logistic Regression**
- **GaussianNB (naive bayes)**
- **Decision Tree**

Procedure:

1. Import the required Python libraries, including Pandas and the necessary Scikit-learn modules.

2. Load the Digits dataset using load_digits () and separate the independent variables X and the target variable y.

3. Display the shape of the dataset and the number of classes.

4. Split the dataset into training and testing sets using train_test_split (), with 80% of the data for training and 20% for testing.

5. Define five classification algorithms: Support Vector Machine (SVM), Random Forest, Logistic Regression, Gaussian Naive Bayes, and Decision Tree.

6. Specify the hyperparameter grids for each classifier to identify the best parameter combinations.

7. Use GridSearchCV () with five-fold cross-validation to find the optimal parameters for each classifier based on accuracy.

8. Fit each grid search model using the training data and predict the target values for the test data.

9. Calculate the test accuracy and record the best parameters and cross-validation accuracy for each classifier.

10. Compare all five classifiers by arranging them in descending order of cross-validation accuracy.

11. Identify the best classifier based on the highest cross-validation accuracy and display its optimal parameters and final test accuracy.

12. Generate a classification report for the best classifier, including precision, recall, F1-score, and support.

Solution:

```
# Compare Classifiers on the Digits Dataset


import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
# 1. Load the Digits dataset
digits = load_digits()
X = digits.data
y = digits.target
print("Dataset shape:", X.shape)
print("Number of classes:", len(set(y)))
# 2. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,random_state=42,stratify=y)
# 3. Define classifiers and their hyperparameter grids
models = {
"SVM": (
Pipeline([
("scaler", StandardScaler()),
("model", SVC())]),
```

```
{
"model__C": [0.1, 1, 10],
"model__kernel": ["linear", "rbf"],
"model__gamma": ["scale", "auto"]}),
"Random Forest": (
RandomForestClassifier(random_state=42),
{
"n_estimators": [50, 100, 200],
"max_depth": [None, 10, 20],
"min_samples_split": [2, 5]}),
"Logistic Regression": (
Pipeline([
("scaler", StandardScaler()),
("model", LogisticRegression(max_iter=5000))]),
{
"model__C": [0.1, 1, 10],
"model__solver": ["lbfgs"]}),
"GaussianNB": (
GaussianNB(),
{
"var_smoothing": [1e-11, 1e-9, 1e-7, 1e-5]}),
"Decision Tree": (
DecisionTreeClassifier(random_state=42),
{
"max_depth": [None, 10, 20],
"min_samples_split": [2, 5, 10],
"criterion": ["gini", "entropy"]})}
# 4. Perform Grid Search for each classifier
results = []
best_models = {}
```

```
for name, (model, param_grid) in models.items():
    grid = GridSearchCV(estimator=model,param_grid=param_grid,
        cv=5,scoring="accuracy",n_jobs=-1)

    grid.fit(X_train, y_train)

    y_pred = grid.predict(X_test)
    test_accuracy = accuracy_score(y_test, y_pred)

    best_models[name] = grid

    results.append({"Classifier": name,
        "Best CV Accuracy": grid.best_score_,
        "Test Accuracy": test_accuracy,
        "Best Parameters": grid.best_params_})

    print("\n", "=" * 50)
    print("Classifier:", name)
    print("Best Parameters:", grid.best_params_)
    print("Best CV Accuracy:", round(grid.best_score_, 4))
    print("Test Accuracy:", round(test_accuracy, 4))

# 5. Compare all classifiers
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(
by="Best CV Accuracy",
ascending=False)
print("\n\nCLASSIFIER PERFORMANCE COMPARISON")
print(results_df[["Classifier", "Best CV Accuracy", "Test
Accuracy"]].to_string(index=False))
```

```
# 6. Identify the best classifier based on CV accuracy
best_name = max(
best_models,
key=lambda name: best_models[name].best_score_)
best_grid = best_models[best_name]
print("\n", "=" * 50)
print("BEST CLASSIFIER:", best_name)
print("OPTIMAL PARAMETERS:", best_grid.best_params_)
print("BEST CV ACCURACY:", round(best_grid.best_score_, 4))
print("FINAL TEST ACCURACY:",
round(accuracy_score(y_test, best_grid.predict(X_test)), 4))
print("\nCLASSIFICATION REPORT")
print(classification_report(
y_test,
best_grid.predict(X_test)))
```

Output:

```
Dataset shape: (1797, 64)
Number of classes: 10

==================================================
Classifier: SVM
Best Parameters: {'model__C': 1, 'model__gamma': 'scale', 'model__kernel': 'rbf'}
Best CV Accuracy: 0.9833
Test Accuracy: 0.975

==================================================
Classifier: Random Forest
Best Parameters: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 200}
Best CV Accuracy: 0.9756
Test Accuracy: 0.9639
```

==========

Classifier: Logistic Regression

Best Parameters: {'model__C': 1, 'model__solver': 'lbfgs'}

Best CV Accuracy: 0.9701

Test Accuracy: 0.9722

==========

Classifier: GaussianNB

Best Parameters: {'var_smoothing': 1e-05}

Best CV Accuracy: 0.8928

Test Accuracy: 0.8722

==========

Classifier: Decision Tree

Best Parameters: {'criterion': 'entropy', 'max_depth': None, 'min_samples_split': 2}

Best CV Accuracy: 0.8636

Test Accuracy: 0.8389

CLASSIFIER PERFORMANCE COMPARISON

Classifier	Best CV Accuracy	Test Accuracy
SVM	0.983297	0.975000
Random Forest	0.975634	0.963889
Logistic Regression	0.970069	0.972222
GaussianNB	0.892835	0.872222
Decision Tree	0.863596	0.838889

==========

BEST CLASSIFIER: SVM

OPTIMAL PARAMETERS: {'model__C': 1, 'model__gamma': 'scale', 'model__kernel': 'rbf'}

BEST CV ACCURACY: 0.9833

FINAL TEST ACCURACY: 0.975

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	36
1	0.95	0.97	0.96	36
2	1.00	1.00	1.00	35
3	1.00	1.00	1.00	37
4	0.95	0.97	0.96	36
5	0.97	1.00	0.99	37
6	0.97	1.00	0.99	36
7	0.92	0.97	0.95	36
8	1.00	0.91	0.96	35
9	1.00	0.92	0.96	36
accuracy			0.97	360
macro avg	0.98	0.97	0.97	360
weighted avg	0.98	0.97	0.97	360